

Guiding End-to-End Driving Models with Endpoint-Constrained Trajectory Optimization

Brayden Zhang¹, Mahsa Golchoubian^{1,2}, Igor Gilitschenski^{1,2}, Boris Ivanovic³, and Kashyap Chitta^{4,5*}

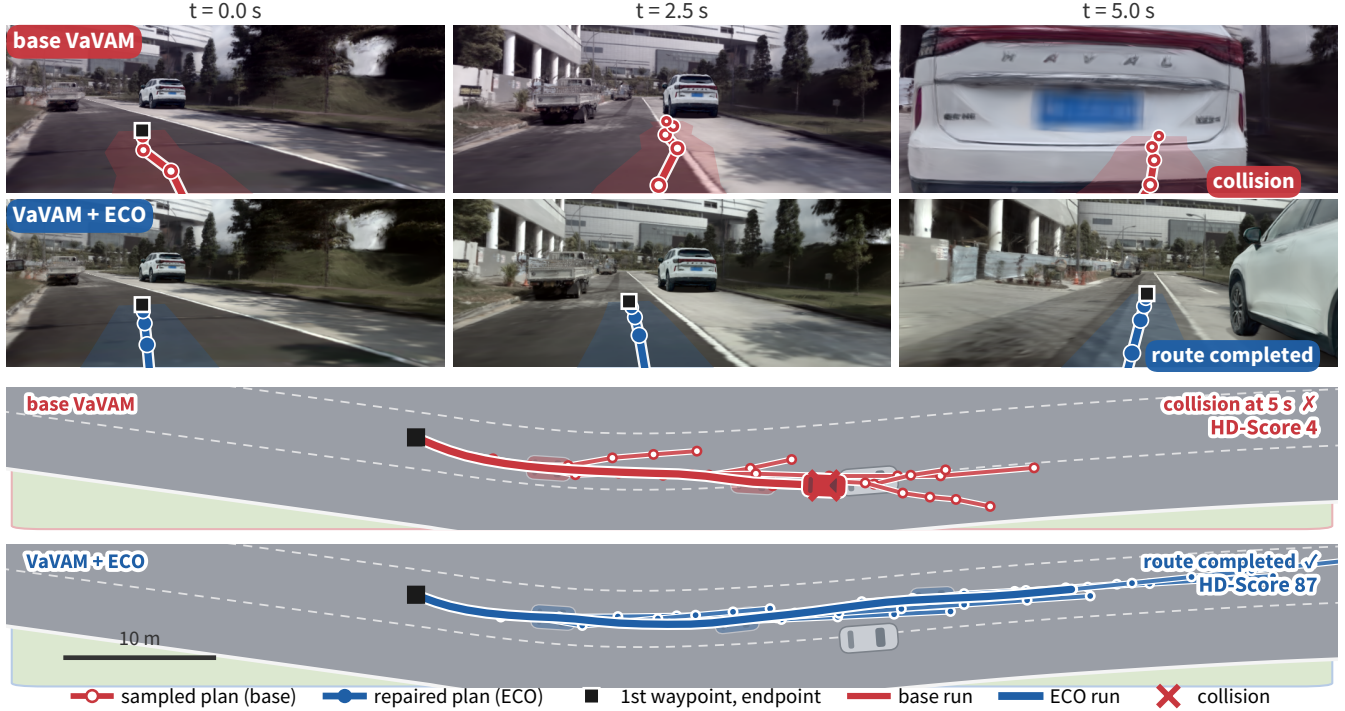


Fig. 1: **Endpoint-Constrained Optimization (ECO)** in HUGSIM. Top: Front-camera views comparing the base VaVAM policy against VaVAM + ECO. At $t = 0$ the sampled base plan (red) exhibits lateral inconsistencies. ECO retains the endpoint and starts from the executed state while optimizing the waypoints between them into a smooth trajectory (blue). In the resulting closed-loop rollouts, base VaVAM later collides, whereas VaVAM + ECO completes the route. Bottom: Bird’s-eye view of the same rollouts, where ECO allows the vehicle to complete the route, achieving a higher HD-Score.

Abstract—End-to-end driving policies are commonly trained through open-loop behavior cloning, yet they must ultimately operate in closed-loop when deployed on a vehicle, creating a fundamental mismatch between training and execution. Beyond the commonly studied effects of covariate shift and causal confusion, we identify a complementary factor for this open-loop/closed-loop gap: waypoint-based supervision and displacement metrics do not ensure that the intermediate trajectory is physically coherent or easy for the controller to track. We observe that these inconsistencies concentrate primarily at intermediate waypoints, while the predicted endpoint remains comparatively reliable. Based on this observation, we introduce Endpoint-Constrained Optimization (ECO), a lightweight post-processing layer that anchors the trajectory to the vehicle’s executed history, preserves the policy’s predicted endpoint, and reshapes the intermediate waypoints to improve feasibility. ECO requires no map, privileged simulator state, or additional train-

ing, and can be inserted between a broad range of waypoint-emitting policies and their controllers. Across two closed-loop simulators, it improves the aggregate closed-loop score of all six evaluated generative and regression-based driving policies, and the gains tend to increase with how often the base plans violate motion limits. On HUGSIM, ECO improves VaVAM from 18.1 to 31.0 HD-Score (+71%), achieving 1st place on the HUGSIM Closed-Loop Driving Challenge. Similarly, on AlpaSim, ECO increases the scene scores of VaVAM and DiffusionDrive by 123% and 22%, respectively. These results show that for a broad collection of end-to-end driving models, repairing the intermediate geometry of predicted trajectories without changing the policy’s predicted endpoint can substantially improve closed-loop performance.

I. INTRODUCTION

Driving policies are increasingly trained end-to-end to predict future trajectories from sensor observations and navigation commands [1]. This is most commonly done through open-loop behavior cloning, where predicted trajectories are

¹University of Toronto, ²Vector Institute, ³NVIDIA Research, ⁴ELLIS Institute Tübingen, ⁵KE:SAI. *Work done in part while at NVIDIA Research. Contact: brayden.zhang@mail.utoronto.ca

Project page: brayden-zhang.github.io/guiding-e2e-eco

compared against logged human driving. In closed-loop evaluation, however, the vehicle executes actions derived from these predictions autoregressively, changing its future observations and subsequent predictions. As a result, a policy may achieve strong open-loop trajectory metrics yet fail during rollouts in closed-loop environments, illustrating the well-known open-loop/closed-loop (OL/CL) gap [2].

Most existing work on this gap has focused on covariate shift and causal confusion [3], [4] as potential causes. However, a complementary factor is that open-loop training uses waypoint-based representations that reward agreement with logged positions without explicitly enforcing the physical plausibility of the output trajectories. Training loss functions are typically applied to all predicted waypoints at an equal scale, leading to a larger learning signal for distant endpoints than nearer intermediate waypoints. Open-loop metrics like average displacement error (ADE) look at waypoint positions in isolation, so they miss the accelerations, jerks, and discontinuities among the waypoints as well as between the vehicle’s historical states and waypoint predictions. Other metrics like final displacement error (FDE) ignore the entire path except the endpoint. As a result, a plan may score strongly in open-loop even if its intermediate waypoints are hard for the controller to follow and extract actions from.

We therefore introduce **Endpoint-Constrained Optimization (ECO)**, a training-free post-processor that improves a policy’s executability by preserving the policy’s high-level plan while refining the intermediate trajectory. Specifically, we apply a lightweight optimization on the sequence of historical and predicted waypoints, with the history and end poses kept fixed, encouraging physical plausibility. ECO consumes and returns the same fixed-period waypoint representation, so it can be inserted between any waypoint-emitting policy and its controller, whether the policy uses a generative [5], [6] or regression-based [7]–[10] head. Across two simulators [11], [12] and five datasets (nuScenes [13], Waymo [14], KITTI-360 [15], PandaSet [16], and the AlpacaSim NuRec set [12]), ECO increases the aggregate closed-loop score of every evaluated policy. These improvements are significant enough to place first in the HUGSIM Closed-Loop Driving Challenge [17] and require no simulator-specific hyperparameter re-tuning, despite differences in vehicle dynamics, controllers, and evaluation settings.

Our contributions are:

- An analysis of the OL/CL gap showing that standard displacement metrics can miss physically inconsistent waypoint sequences, with most violations occurring in the middle of the trajectory rather than at its endpoints.
- ECO, a training-free trajectory optimization layer that keeps the executed history and the policy’s endpoint fixed and repairs the trajectory between them.
- A set of controlled closed-loop studies that isolate why this refinement works and when it transfers.

II. RELATED WORK

Closed-loop evaluation of end-to-end driving. Photorealistic neural simulators such as HUGSIM [11] and Al-

paSim [12] now make it significantly easier to study how policies behave under online interaction, rather than only through offline trajectory metrics. Even on these new photorealistic simulators, our work shows that behavior cloning for end-to-end driving suffers from a discrepancy between offline evaluation and actual closed-loop performance, in line with earlier work [2], [3], [18]. In particular, recent end-to-end driving policies increasingly use generative planning heads based on diffusion [6] and flow matching [19] to model complex multi-modal behaviors. We observe a large open-loop/closed-loop gap for this now predominant generative planning paradigm.

Improving closed-loop robustness. Existing approaches address closed-loop failures at several levels. Interactive data collection [18] (either in real life or simulation) and simulation-based closed-loop fine-tuning [20] modify the policy using feedback from expert labels or closed-loop rewards, respectively. Other work analyzes biases in open-loop training and evaluation to improve closed-loop robustness. For example, driving models can learn shortcuts such as target point following [21], which can be mitigated in open-loop training to improve closed-loop performance. In a similar spirit, we analyze the impact of physical implausibility stemming from open-loop training using waypoint prediction losses. Our proposed ECO module leaves the policy’s training unchanged and instead operates at the policy–controller interface, improving closed-loop executability.

Trajectory refinement and post-processing. Trajectory optimization has long been used to refine candidate motions under geometric and dynamical constraints. Classical methods include CHOMP [22] and TrajOpt [23], while driving-specific approaches include Frenet-frame generation [24] and path–velocity decomposition [25]. More recent driving systems also account explicitly for downstream execution. For example, PDM-Closed [26] simulates rule-based trajectory proposals through a vehicle model and tracking controller before selecting among them, while safety filters and control barrier functions intervene at the control level to enforce constraints [27], [28]. Other approaches modify or select learned trajectories, including safety-oriented planners and proposal-selection schemes [29], [30], or apply conditioning/constraints to guide diffusion-based trajectory models [31]–[33]. Closer to our setting, Hydra-NeXt [34] learns a trajectory-refinement module jointly with trajectory and control prediction to improve kinematic consistency, while TOAD [35] performs test-time trajectory search using a learned trajectory scorer. In contrast, ECO requires neither additional training nor a learned scorer, and operates only on the policy’s emitted trajectory and execution history while preserving the predicted endpoint. Despite this simplicity, it shows strong effectiveness across simulators and policies.

III. PROBLEM FORMULATION

Closed-loop policy–controller interface. A pre-trained end-to-end driving policy receives images and a navigation command from its environment and predicts N waypoints

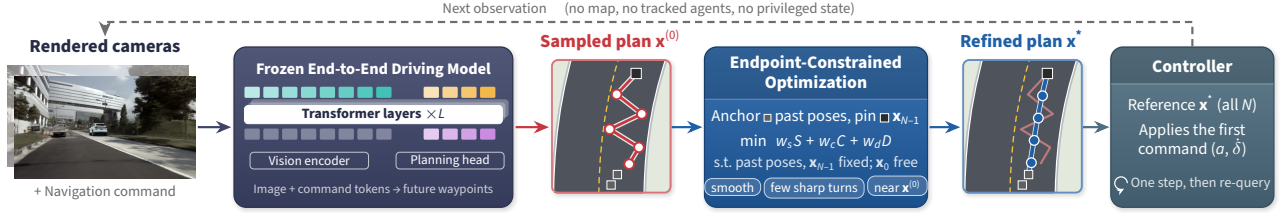


Fig. 2: **Pipeline.** The frozen end-to-end driving model receives camera inputs from the simulator and samples a trajectory for the next N steps. ECO anchors $K = 2$ past poses and the last predicted waypoint, then solves a trajectory optimization problem to refine the waypoints between them (Sec. IV). The controller then tracks the refined plan.

at Δt intervals in the ego frame. A trajectory-tracking controller [36] uses the planned trajectory as its reference to extract a control sequence, applies the first action command in this sequence to reach a new environment state, and then queries the policy again.

Predicted waypoint sequence. The policy emits a predicted plan $\mathbf{x}^{(0)} = (\mathbf{x}_0^{(0)}, \dots, \mathbf{x}_{N-1}^{(0)})$ of N waypoints at some interval Δt , where N is determined by the underlying policy. Each waypoint is $\mathbf{x}_i = (\mathbf{p}_i, \theta_i)$, with position $\mathbf{p}_i = (x_i, y_i)$ and heading θ_i . For policies that predict positions only, we define θ_i as the heading of the segment from $\mathbf{p}_{i-1}$ to $\mathbf{p}_i$, with $\mathbf{p}_{-1}$ at the ego origin. We write the vehicle’s executed poses in the same $(\mathbf{p}, \theta)$ form as $\mathbf{h}_{-K}, \dots, \mathbf{h}_{-1}$, with the current pose $\mathbf{h}_0 = (0, 0, 0)$ at the origin of the ego frame.

Post-inference trajectory optimization. We apply an optimization step after inference and before control that maps the predicted plan to an optimized plan $\mathbf{x}^*$, which becomes the controller’s reference with the same horizon and timing. The base policy remains unchanged, and the optimization itself only sees the emitted plan and executed ego history.

Problem. Which parts of the emitted plan should such a post-processor be allowed to change? Sec. V-B shows that motion-limit violations concentrate in the plan interior; we therefore ask whether repairing the intermediate waypoints $\mathbf{x}_{0:N-2}$, while keeping the last waypoint fixed, improves closed-loop performance in terms of route completion, driving safety, and comfort. Sec. IV instantiates this as ECO.

IV. ENDPOINT-CONSTRAINED OPTIMIZATION

ECO is placed between the policy and controller (Fig. 2). Before optimization, we prepend the two most recently executed poses $\mathbf{h}_{-2:-1}$ and the current pose $\mathbf{h}_0 = (0, 0, 0)$ to the predicted trajectory. We hold these historical poses and the final predicted waypoint $\mathbf{x}_{N-1}$ fixed, while optimizing the remaining waypoints $\mathbf{x}_{0:N-2}$. Each historical pose $\mathbf{h}_{-k}$ represents the vehicle pose $k\Delta t$ before the current query, with Δt matching the prediction interval. We obtain these poses by interpolating the executed ego trajectory and transforming them into the current ego frame. At the beginning of an episode, when sufficient history is not yet available, we anchor only the executed poses that exist, starting with the current pose. Because the objective is evaluated over the combined executed-and-predicted trajectory, its finite differences encourage a smooth transition into the new plan.

TABLE I: **Components of Eq. (1).** Indices run over the extended trajectory $\mathbf{z}$ (Sec. IV).

Term	Definition
Smoothness $S(\mathbf{z})$	$\sum_{i=-K-1}^{N-3} \left[\mathbf{p}_{i+2} - 2\mathbf{p}_{i+1} + \mathbf{p}_i ^2 + \text{wrap}(\theta_{i+2} - 2\theta_{i+1} + \theta_i)^2 \right]$
Turn penalty $C(\mathbf{z})$	$\sum_{i=-K}^{N-2} \mathbf{1}_i (c_i - c_0)^2, \quad c_i = (\mathbf{p}_i - \mathbf{p}_{i-1}) \times (\mathbf{p}_{i+1} - \mathbf{p}_i)$
Deviation $D(\mathbf{x}, \mathbf{x}^{(0)})$	$\sum_{i=0}^{N-1} \mathbf{x}_i - \mathbf{x}_i^{(0)} ^2$

Objective. We optimize the intermediate waypoints using an objective that balances trajectory smoothness, turn sharpness, and fidelity to the predicted plan. These terms are evaluated on the extended trajectory $\mathbf{z} = (\mathbf{h}_{-K}, \dots, \mathbf{h}_0, \mathbf{x}_0, \dots, \mathbf{x}_{N-1})$ (Table I), in which the anchored poses $\mathbf{h}_{-K:0}$ are fixed parameters, and the deviation term is applied on the plan alone. We optimize:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} w_s S(\mathbf{z}) + w_c C(\mathbf{z}) + w_d D(\mathbf{x}, \mathbf{x}^{(0)}) \quad (1)$$

$$\text{s.t. } \mathbf{x}_{N-1} = \mathbf{x}_{N-1}^{(0)}. \quad (2)$$

Table I defines the three objective terms, with $\mathbf{z}_i = \mathbf{h}_{i+1}$ for $i < 0$ and $\mathbf{z}_i = \mathbf{x}_i$ for $i \geq 0$. The smoothness term S penalizes second differences in position and heading, discouraging abrupt changes between consecutive waypoints; angular differences are wrapped to $[-\pi, \pi]$. The turn penalty C penalizes sharp turns using the cross product c_i of consecutive trajectory segments. We apply this penalty only when $|c_i| > c_0 = 0.5 \text{ m}^2$ and both segments are longer than 0.1 m, avoiding unstable penalties near stationary points. Finally, the deviation term D discourages the optimized trajectory from moving too far from the policy’s original prediction. Positions and headings are jointly optimized.

Optimization. We solve Eq. (1) with L-BFGS-B [37], initialized at the predicted plan. On a Xeon Gold 6238R CPU, the algorithm takes a median of 31 iterations and 20.7 ms to solve. This is comfortably within a typical end-to-end planner’s inference budget, allowing parallelized execution and pipelining (e.g., the inference of VaVAM [5] takes 44.7 ms forward pass on an L40S GPU). It also comfortably fits within the assigned 250 ms and 100 ms control periods for the HUGSIM and AlpaSim simulators, respectively.

TABLE II: **Small open-loop changes can produce large closed-loop gains.** Results on the 34-scene nuScenes subset of the HUGSIM test set. ADE/FDE compare the plan to the ground truth, whereas paired Δ measures the change from each raw plan to its ECO-processed version. Deficit is the fraction of waypoints violating a motion limit. ECO changes paired ADE/FDE by only centimeters while improving HD-Score by +16.5 for VaVAM and +9.5 for UniAD. The ADE/FDE columns are computed over the plans encountered in each closed-loop rollout and therefore reflect different visited-state distributions. Paired Δ ADE/ Δ FDE instead compares each raw plan with its ECO-processed version at the same state, isolating the direct effect of post-processing.

Sent plan	open loop (m)		paired Δ after ECO		closed loop
	ADE $\downarrow$	FDE $\downarrow$	Δ ADE / Δ FDE (m)	deficit [%] $\downarrow$	HD-Score $\uparrow$
VaVAM	2.53	3.27	–	44	23.1
VaVAM + ECO	1.66	2.42	–0.03 / +0.00	0	39.6
UniAD	1.86	2.25	–	16	27.8
UniAD + ECO	2.02	2.61	+0.03 / +0.00	0	37.3

V. EXPERIMENTS

Our experiments address three questions: where trajectory inconsistencies arise, whether ECO improves closed-loop driving across policies and simulators, and which components drive the gains. We first describe the experimental setup, then analyze trajectory errors, evaluate ECO, and conclude with controlled ablations.

A. Experimental Setup

Benchmarks and evaluation splits. Our primary closed-loop benchmark is HUGSIM [11], a photorealistic closed-loop simulator built from 3D Gaussian Splatting [38] reconstructions of real-world driving logs from nuScenes [13], Waymo [14], KITTI-360 [15], and PandaSet [16]. HUGSIM provides reactive traffic across four difficulty levels, with the hard and extreme modes introducing aggressive actors that may perform adversarial, collision-seeking maneuvers. The HUGSIM dataset has two splits: a 345-scene “train” split released before the associated closed-loop driving challenge, and a 185-scene “test” split released afterward. While ECO does not involve any training, we maintain this naming convention throughout the results. For ablation studies, we use the nuScenes subset of the HUGSIM test split. We additionally evaluate in AlpaSim [12], a neural reconstruction-based closed-loop simulator from NVIDIA. It runs its own traffic simulation and vehicle dynamics, and tracks the policy’s plan with a model-predictive controller rather than HUGSIM’s iLQR-based controller. For evaluation, following [39], we use the 441-scene NuRec validation set of the AlpaSim end-to-end driving challenge.

Policies and controllers. Our primary analysis uses VaVAM [5], which combines a video-generative backbone pre-trained on OpenDV-2K [40] with a flow-matching [19] action head post-trained on nuPlan [41] and nuScenes [13]; “VaVAM” denotes the released VaVAM-B checkpoint

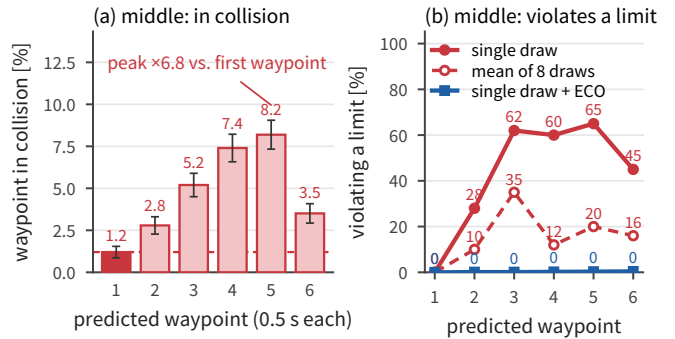


Fig. 3: **Collisions and limit violations concentrate in the middle of a predicted plan.** Measured with VaVAM, on the nuScenes subset of the HUGSIM test set. (a) Fraction of predicted waypoints whose ego footprint overlaps an agent box recorded in the rollout at the frame each waypoint is meant for (i.e., waypoints in collision). (b) Waypoints violating a motion limit for a single sampled plan, for the average of eight sampled plans, and for one plan after ECO.

throughout, and VaVAM-S/-L (Fig. 5) are the released smaller and larger checkpoints of the same architecture and training recipe. We evaluate ECO in closed loop using HUGSIM and AlpaSim with their default controllers, unless otherwise mentioned. Both the policy sampler and the traffic simulation can be stochastic: the action head samples noise at each control step, while the hard and extreme HUGSIM scenes include stochastic agent behavior.

Baselines. We evaluate a set of diverse planning architectures with ECO. In addition to the flow-matching-based VaVAM, we also use the truncated diffusion-based DiffusionDrive [6] as well as several deterministic or proposal-based planners, namely DrivoR [10], UniAD [7], VAD [8], and Latent TransFuser (LTF) [9]. Unless otherwise specified, all methods use the same scenes, controller, and evaluation protocol. As post-processing baselines, we replace ECO with alternative trajectory-shaping operators under the same endpoint constraints (Sec. V-D).

Metrics and evaluation protocol. Within HUGSIM, the primary HD-Score metric combines route completion R_c with a per-step score in which no-collision (NC) and drivable-area (DAC) act as gates and time-to-collision (TTC) and comfort (COM) contribute to the remaining score. For the exact equations, please refer to [11]. For AlpaSim, following the setting of [39], we use the standard metrics of the e2e_challenge branch. These metrics are frozen at the first at-fault collision, off-road event, or exit from the 4 m corridor around the logged route. The scene score rewards route progress above 80% for scenes without these failures, while m/incident is the distance driven per at-fault collision or off-road event, in meters. We also report route progress and the corresponding failure rates. For the detailed analysis of trajectory feasibility, we define a waypoint as *violating a limit* when the finite differences of the predicted trajectory exceed one of the HUGSIM motion limits [11]: yaw rate,

yaw acceleration, longitudinal acceleration, and jerk, with thresholds of 0.95 rad/s, 1.93 rad/s², $[-4.05, 4.89]$ m/s², and 8.37 m/s³, respectively. We define the *kinematic deficit* of a plan or policy as the fraction of its waypoints that violate at least one of these limits. Note that we use these limits only for evaluation, not in Eq. (1), and hold them fixed across policies and simulators.

Implementation details. We select $w_s = 3$, $w_c = 8$, and $w_d = 0.05$ using a coarse hyperparameter sweep on a small held-out subset of the HUGSIM train split. We chose the setting that reduced kinematic-limit violations while maintaining fidelity to the original predicted trajectory, and subsequently froze these weights for all reported policies, HUGSIM test experiments, and AlpaSim evaluations. These parameters are relatively insensitive to moderate value changes; scaling the smoothness, turn penalty, or deviation weight by $3\times$ or $1/3\times$ changes HD-Score by no more than 2.4 points. The only exception is when the deviation penalty is increased beyond $w_d = 0.5$ to hold the plan to the original trajectory and prevent the optimization almost entirely.

B. Where Errors Arise Along Planned Trajectories

Open-loop displacement is misaligned with closed-loop success. For each predicted plan, we first calculate the mean waypoint displacement from the relevant human trajectory. Across all the episodes driven by VaVAM in the nuScenes subset of HUGSIM, plans from collision episodes are *closer* to the human driving path than those from completed episodes, with median displacements of 1.2 and 2.0 m, respectively. Hence, merely having good geometric agreement with the demonstration does not show whether a predicted trajectory will be easy to carry out in a closed loop.

Table II shows similar patterns across two policies on the same HUGSIM nuScenes split. ECO changes the predicted trajectories relative to the human route by only centimeters and, for UniAD, slightly *away* from it (higher ADE), while the closed-loop score on the same episodes increases by +16.5 points for VaVAM and +9.5 points for UniAD.

Failure concentrates in the interior. We next examine how trajectory errors vary along the prediction horizon. At each predicted waypoint, we place the ego vehicle’s spatial footprint at the predicted pose and test for overlap with agent boxes from the corresponding future frame of the same rollout, using the final frame for waypoints beyond the episode. The intersection rate rises from 1.2% at the first predicted waypoint to 8.2% at 2.5 s, before dropping to 3.5% at the endpoint (Fig. 3a). We observe the same pattern for motion-limit violations, which occur predominantly at intermediate waypoints (Fig. 3b), rather than at the boundaries of the prediction horizon. Together, these analyses indicate that the largest concentration of undesirable behavior lies in the intermediate trajectory waypoints.

C. ECO Improves Closed-Loop Driving

Main results. ECO substantially improves closed-loop performance on the full HUGSIM test set (Table III, Fig. 4).

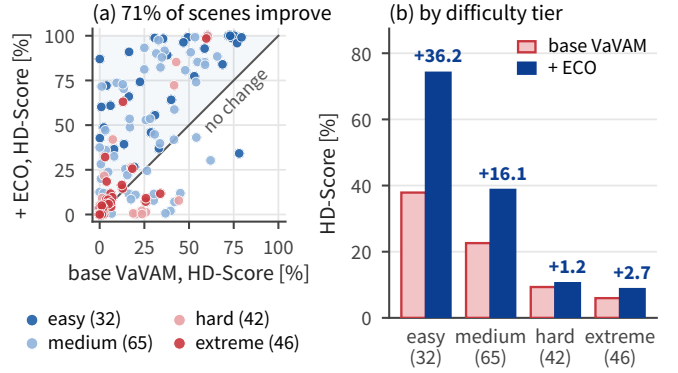


Fig. 4: **Results by HUGSIM difficulty tier.** (a) Every scene of the 185-scene test set as one point: HD-Score without and with ECO. Points above the diagonal improve. (b) ECO helps most on easier scenes where the base policy is already competent.

For VaVAM, the HD-Score increases from 18.1 to 31.0, the largest gain among all tested policies. The gain is not limited to comfort: Route completion for VaVAM increases from 40.9 to 45.2. The improvement is not concentrated in a small number of scenes: 71% of the 185 scenes improve (Fig. 4a), with a median per-scene gain of +3.3. The magnitude varies with scene difficulty (Fig. 4b): ECO improves HD-Score by +36.2, +16.1, +1.2, and +2.7 on easy, medium, hard, and extreme scenes, respectively. The smaller gains on the hardest scenes suggest that ECO alone cannot address all failure modes, particularly those that would require a completely different planned endpoint to solve the scenario. The gains broadly match how much of the plan the objective can repair: gains are large for VaVAM, moderate for UniAD, and smaller for LTF, VAD, and DrivoR, whose plans already satisfy most motion limits. Additionally, the gain is not limited to the test split; compared to existing baselines on the 345-scene train split, ECO improves VaVAM from 20.0 to 32.1 and DrivoR (w/ SimScale [42]) from 35.7 to 38.8 (Table IV).

Further evaluation in AlpaSim. We next run VaVAM + ECO in AlpaSim [12] (Table III, bottom), with both the default nonlinear MPC controller and an alternative linear MPC controller. We find similar gains as in HUGSIM, despite different controller types. Furthermore, AlpaSim contains no comfort score, yet ECO also improves its aggregate closed-loop metrics, providing additional evidence that the gain from ECO is not limited to comfort. We also test DiffusionDrive [6] as a second generative policy which improves along every reported AlpaSim metric with ECO.

Model sizes and generation steps. Across 10 policy and sampler variants analyzed in Fig. 5, larger deficits correspond to larger closed-loop gains from ECO (Spearman $\rho = 0.64$, $p = 0.0479$). Policies whose waypoint sequences contain more kinematic inconsistencies leave more room for post-inference refinement. For generative heads, ECO also provides a favorable speed–performance trade-off. A 2-step VaVAM sampler with ECO reaches 38.7 HD-Score on the

TABLE III: **Cross-policy and cross-simulator evaluation of ECO.** Scores are percentages unless a unit is shown. $\pm$ is the standard deviation over passes; Δ rows combine the base and ECO deviations in quadrature. *Top*: HD-Score on the 185-scene HUGSIM test set by domain and overall, with route completion, no-collision, TTC, and comfort. *Bottom*: Evaluation in AlpaSim, an independent simulator with no comfort score; lin./nonlin. MPC: AlpaSim’s linear/nonlinear model-predictive controller. ECO consistently improves the aggregate scores of all base policies in both simulators.

HUGSIM [11] Test Set (185 scenes), mean over 3 evaluation seeds									
Policy	HD-Score $\uparrow$					$R_c \uparrow$	NC $\uparrow$	TTC $\uparrow$	COM $\uparrow$
	Waymo	nuScenes	KITTI-360	PandaSet	Overall				
VaVAM [5]	25.8	23.1	4.0	23.1	18.1 $\pm$ 0.5	40.9	52	38	14
VaVAM + ECO	40.0	39.6	22.2	30.3	31.0$\pm$0.6	45.2	54	46	97
Δ (ECO – base)	+14.2	+16.5	+18.2	+7.2	+12.9 $\pm$ 0.8	+4.3	+2	+8	+83
LTF [9]	34.5	17.8	3.7	22.6	17.9 $\pm$ 1.1	31.6	44	39	100
LTF + ECO	29.5	31.9	8.2	21.6	20.7$\pm$0.3	33.2	38	36	100
Δ (ECO – base)	–5.0	+14.1	+4.5	–1.0	+2.8 $\pm$ 1.1	+1.6	–6	–3	0
UniAD [7]	27.5	27.8	2.9	42.6	26.8 $\pm$ 0.8	36.7	69	57	72
UniAD + ECO	28.5	37.3	8.1	48.0	32.4$\pm$0.8	39.8	72	63	100
Δ (ECO – base)	+1.0	+9.5	+5.2	+5.4	+5.6 $\pm$ 1.1	+3.1	+3	+6	+28
VAD [8]	8.2	23.8	1.1	24.4	15.7 $\pm$ 0.9	26.6	55	43	100
VAD + ECO	11.0	28.6	1.9	24.9	17.4$\pm$0.4	29.4	52	39	100
Δ (ECO – base)	+2.8	+4.8	+0.8	+0.5	+1.7 $\pm$ 1.0	+2.8	–3	–4	0
DrivoR (w/ SimScale [42]) [10]	39.0	47.2	18.1	38.9	34.6 $\pm$ 0.8	44.0	63	58	97
DrivoR (w/ SimScale) + ECO	43.9	48.0	21.6	38.7	36.2$\pm$0.3	46.0	62	58	96
Δ (ECO – base)	+4.9	+0.8	+3.5	–0.2	+1.6 $\pm$ 0.9	+2.0	–1	0	–1
AlpaSim [12] NuRec validation set (441 scenes)									
Policy	Scene score $\uparrow$	route prog. $\uparrow$	dist./route $\uparrow$	m/incident $\uparrow$	in corridor $\uparrow$	off-road $\downarrow$	at-fault coll. $\downarrow$		
VaVAM (nonlin. MPC)	11.0	42.9	65.6 m	143 m	57.6	28.0	18.0		
VaVAM (nonlin. MPC) + ECO	24.5	51.4	82.4 m	184 m	67.1	29.7	15.2		
Δ (ECO – base)	+13.5	+8.5	+16.8 m	+41 m	+9.5	+1.7	–2.8		
VaVAM (lin. MPC)	32.8	60.8	112.1 m	353 m	74.8	15.0	16.8		
VaVAM (lin. MPC) + ECO	39.2	63.1	116.0 m	390 m	79.8	16.8	12.9		
Δ (ECO – base)	+6.4	+2.3	+3.9 m	+37 m	+5.0	+1.8	–3.9		
DiffusionDrive [6] (nonlin. MPC)	28.9	40.6	59.8 m	190 m	77.8	22.2	9.3		
DiffusionDrive (nonlin. MPC) + ECO	35.2	43.5	67.0 m	296 m	81.7	17.1	5.5		
Δ (ECO – base)	+6.3	+2.9	+7.2 m	+106 m	+3.9	–5.1	–3.8		

TABLE IV: **HD-Score on both HUGSIM splits.** Train (345 scenes) and test (185 scenes); *taken from [10].

	VaVAM [5]		UniAD [7]		DrivoR (w/ SimScale [42]) [10]	
	Train	Test	Train	Test	Train	Test
base policy	20.0	18.1	32.7*	26.8	35.7*	34.6
+ ECO	32.1	31.0	34.9	32.4	38.8	36.2

nuScenes split of the HUGSIM test set, compared with 21.3 for the 20-step base sampler, while reducing per-plan latency from 65 ms to 49 ms. This suggests that trajectory repair can compensate for coarser generative sampling, enabling cheaper inference while improving closed-loop performance.

D. What Causes the Improvement?

Boundary conditions are important. ECO achieves a 39.6 HD-Score when both the executed history and predicted endpoint are fixed (see Table V), compared to 23.1 for the

TABLE V: **Boundary conditions.** VaVAM with the ECO objective on the nuScenes split of the HUGSIM test set. Plans are either anchored to the executed history $\mathbf{h}_{-K:0}$ (first predicted waypoint $\mathbf{x}_0$ free) or pinned at $\mathbf{x}_0$, with the endpoint $\mathbf{x}_{N-1}$ either free or pinned.

Start boundary	Endpoint $\mathbf{x}_{N-1}$	HD-Score	Δ vs. ECO
$\mathbf{x}_0$ pinned	free	32.1	–7.5
history anchored	free	34.4	–5.2
$\mathbf{x}_0$ pinned	pinned	36.9	–2.7
history anchored	pinned	39.6	–
base policy		23.1	–16.5

base policy. If we instead fix the first predicted waypoint $\mathbf{x}_0$ and the endpoint, but do not anchor the executed history, the score drops to 36.9. If the endpoint is not fixed, the score falls to 34.4 with history anchoring and 32.1 when $\mathbf{x}_0$ is

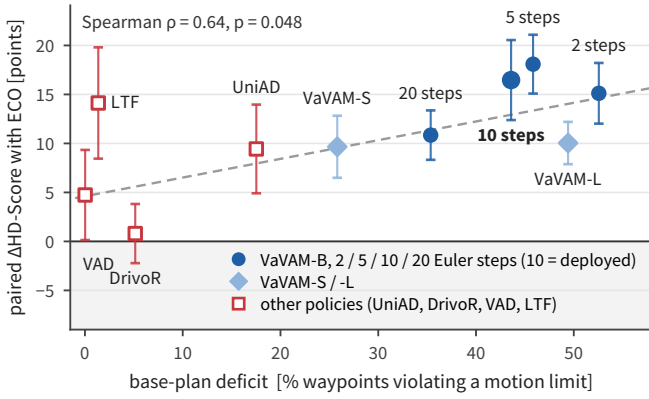


Fig. 5: **ECO benefit tends to increase with base-plan deficit.** Paired Δ HD-Score with ECO against the fraction of base-plan waypoints that violate a motion limit, for VaVAM-B (357M parameters) at 2, 5, 10 (deployed), and 20 flow matching steps, VaVAM-S (204M parameters), VaVAM-L (1.4B parameters), and four other policies on the same nuScenes test split in HUGSIM.

fixed. The first-waypoint-fixed variant therefore provides a substantial boost already, but the full ECO setup accounts for the final 2.7 points of performance.

Choice of objective. Next, we test whether the improvement depends on the specific ECO objective (Fig. 6). Keeping the same boundary conditions (executed history anchored, endpoint pinned, first predicted waypoint free), we replace ECO with a least-squares cubic smoothing spline (smoothing factor $s = N$), Savitzky–Golay filtering [43] (window 5, order 2), a 3-point moving average, a constrained bicycle-model optimizer [44], and temporal ensembling of consecutive plans as in action chunking [45]. These methods reach HD-Scores of 28.9, 31.9, 31.7, 34.8, and 27.8, respectively, compared with 39.6 for ECO. Simply replacing the interior before the policy-selected endpoints with any intuitive approach already recovers part of the lost closed-loop performance, although ECO gives the largest improvement.

Simplified tracking. We also replay HUGSIM’s controller on 1,500 paired base and ECO trajectories from the same ego state in HUGSIM. The controller cost, i.e. the iLQR objective (the sum over the tracking horizon of the quadratic state error to the reference and the quadratic control effort), falls by 8.5%, from 101.2 to 92.6, and is lower for 83% of the pairs. The percentage of steps that reach the steering-rate limit drops from 20.1% to 12.9%. At the same time, the error in reaching the first predicted waypoint barely changes (base: 0.45 m, ECO: 0.44 m). This suggests that ECO’s benefit comes from a more trackable reference over the whole horizon, and not specifically from easier tracking of the first waypoint.

E. Limitations

Because ECO fixes the policy’s endpoint, it is limited in what it can correct. For example, if the endpoint is in a poor position, ECO cannot make the vehicle add a stop the policy

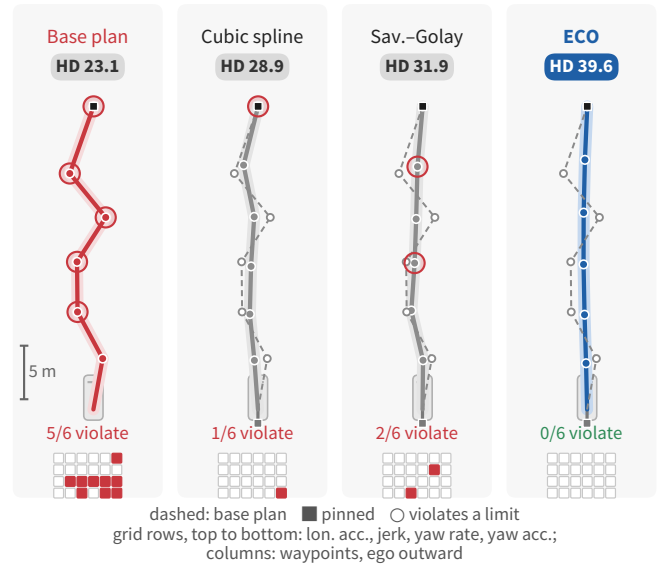


Fig. 6: **Trajectory-shaping operators on a sampled plan.** All operators are applied with the same boundary conditions as ECO: the executed history is anchored, the endpoint is pinned (square), and the first predicted waypoint is free. Rings denote motion-limit violations; HD-Scores are the averages on the nuScenes split (Sec. V-A).

did not predict. This matters especially for collisions: ECO does not reason about other agents (Sec. IV, Fig. 2), so a separate safety layer may still be necessary.

VI. CONCLUSION

In this paper, we investigate the OL/CL gap through the executability of a policy’s plan, and introduce ECO, a lightweight, training-free trajectory optimization layer that keeps the policy’s endpoint fixed, anchors the vehicle’s executed history, and corrects the trajectory between them. ECO yields consistent gains in closed-loop driving across various policies and simulators. More broadly, our results suggest that closing the OL/CL gap requires considering not only what a driving policy predicts but also whether the waypoint sequence it exposes to the controller is within the motion limits the controller can follow. ECO is a simple first step that addresses trajectory-level failures at test time while also complementing methods that can improve the policy’s high-level planner.

ACKNOWLEDGMENT

Claude was used to assist with Matplotlib and LaTeX code generation for Figs. 1, 2, 5, and 6, as well as Python code editing for simulator integration and policy adaptation; all outputs were reviewed and validated by the authors.

REFERENCES

- [1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE TPAMI*, 2024.
- [2] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, “On offline evaluation of vision-based driving models,” in *ECCV*, 2018.

- [3] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *NeurIPS*, 2019.
- [4] P. Karkus, M. Igl, Y. Chen, K. Chitta, J. Packer, B. Douillard, R. Tian, A. Naumann, G. Garcia-Cobo, S. Tan, A. Degirmenci, A. Popov, N. Smolyanskiy, U. Muller, B. Ivanovic, and M. Pavone, "Beyond behavior cloning in autonomous driving: A survey of closed-loop training techniques," NVIDIA Research, Dec. 2025, https://research.nvidia.com/publication/2025-12_beyond-behavior-cloning-autonomous-driving-survey-closed-loop-training.
- [5] F. Bartoccioni, E. Ramzi, V. Besnier, S. Venkataramanan, T.-H. Vu, Y. Xu, L. Chambon, S. Gidaris, S. Odabas, D. Hurych, R. Marlet, A. Boulch, M. Chen, É. Zablocki, A. Bursuc, E. Valle, and M. Cord, "VaViM and VaVAM: Autonomous driving through video generative modeling," *arXiv preprint arXiv:2502.15672*, 2025.
- [6] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, and X. Wang, "DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving," in *CVPR*, 2025.
- [7] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *CVPR*, 2023.
- [8] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "VAD: Vectorized scene representation for efficient autonomous driving," in *ICCV*, 2023.
- [9] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "TransFuser: Imitation with transformer-based sensor fusion for autonomous driving," *IEEE TPAMI*, 2023.
- [10] E. Kirby, A. Boulch, Y. Xu, Y. Yin, G. Puy, É. Zablocki, A. Bursuc, S. Gidaris, R. Marlet, F. Bartoccioni, A.-Q. Cao, N. Samet, T.-H. Vu, and M. Cord, "Driving on registers," in *CVPR*, 2026.
- [11] H. Zhou, L. Lin, J. Wang, Y. Lu, D. Bai, B. Liu, Y. Wang, A. Geiger, and Y. Liao, "HUGSIM: A real-time, photo-realistic and closed-loop simulator for autonomous driving," *arXiv preprint arXiv:2412.01718*, 2024.
- [12] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. G. Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. M. Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, "AlpaSim: A modular, lightweight, and data-driven research simulator for autonomous driving," <https://github.com/NVlabs/alpasim>, Oct. 2025.
- [13] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," in *CVPR*, 2020.
- [14] P. Sun, H. Kretschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov, "Scalability in perception for autonomous driving: Waymo open dataset," in *CVPR*, 2020.
- [15] Y. Liao, J. Xie, and A. Geiger, "KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D," *IEEE TPAMI*, vol. 45, no. 3, pp. 3292–3310, 2023.
- [16] P. Xiao, Z. Shao, S. Hao, Z. Zhang, X. Chai, J. Jiao, Z. Li, J. Wu, K. Sun, K. Jiang, Y. Wang, and D. Yang, "PandaSet: Advanced sensor suite dataset for autonomous driving," in *ITSC*, 2021.
- [17] Y. Liao, H. Zhou, Y. Lu, B. Liu, H. Zhang, J. Wei, Z. Ni, Y. Li, and A. Geiger, "RealADSim workshop at ICCV 2025, track 2: Closed-loop driving in photorealistic simulation (HUGSIM closed-loop driving challenge)," <https://realadsim.github.io/2025/>, 2025.
- [18] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *AISTATS*, 2011.
- [19] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *ICLR*, 2023.
- [20] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering your diffusion policy with latent space reinforcement learning," *arXiv preprint arXiv:2506.15799*, 2025.
- [21] B. Jaeger, K. Chitta, and A. Geiger, "Hidden biases of end-to-end driving models," in *ICCV*, 2023.
- [22] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *ICRA*, 2009.
- [23] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *International Journal of Robotics Research (IJRR)*, vol. 33, no. 9, pp. 1251–1270, 2014.
- [24] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a Frenet frame," in *ICRA*, 2010.
- [25] K. Kant and S. W. Zucker, "Toward efficient trajectory planning: The path-velocity decomposition," *International Journal of Robotics Research (IJRR)*, vol. 5, no. 3, pp. 72–89, 1986.
- [26] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, "Parting with misconceptions about learning-based vehicle motion planning," in *CoRL*, 2023.
- [27] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *European Control Conference (ECC)*, 2019.
- [28] K.-C. Hsu, H. Hu, and J. F. Fisac, "The safety filter: A unified view of safety-critical control in autonomous systems," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, pp. 47–72, 2024.
- [29] M. Vitelli, Y. Chang, Y. Ye, A. Ferreira, M. Wolczyk, B. Osiński, M. Niendorf, H. Grimm, Q. Huang, A. Jain, and P. Ondruska, "SafetyNet: Safe planning for real-world self-driving vehicles using machine-learned policies," in *ICRA*, 2022.
- [30] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, Y.-G. Jiang, and J. M. Alvarez, "Hydra-MDP: End-to-end multimodal planning with multi-target hydra-distillation," *arXiv preprint arXiv:2406.06978*, 2024.
- [31] K. Kondo, A. Tagliabue, X. Cai, C. Tewari, O. Garcia, M. Espitia-Alvarez, and J. P. How, "CGD: Constraint-guided diffusion policies for UAV trajectory planning," *arXiv preprint arXiv:2405.01758*, 2024.
- [32] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus, "SafeDiffuser: Safe planning with diffusion probabilistic models," in *ICLR*, 2025.
- [33] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *ICML*, 2022.
- [34] Z. Li, S. Wang, S. Lan, Z. Yu, Z. Wu, and J. M. Alvarez, "Hydra-NeXt: Robust closed-loop driving with open-loop training," in *ICCV*, 2025.
- [35] Y. Xu, É. Zablocki, Y. Yin, E. Ramzi, E. Kirby, A. Boulch, and M. Cord, "Test-time trajectory optimization for autonomous driving," *arXiv preprint arXiv:2606.07170*, 2026.
- [36] W. Li and E. Todorov, "Iterative linear quadratic regulator design for nonlinear biological movement systems," in *ICINCO*, 2004.
- [37] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Trans. Math. Softw.*, vol. 23, no. 4, pp. 550–560, 1997.
- [38] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
- [39] D. Da Col, M. Igl, P. Karkus, K. Chitta, B. Ivanovic, M. Pavone, K. Schindler, and C. Sakaridis, "OPTED: On-policy fine-tuning for end-to-end driving using a render-free teacher," *arXiv preprint arXiv:2609.20756*, 2026.
- [40] J. Yang, S. Gao, Y. Qiu, L. Chen, T. Li, B. Dai, K. Chitta, P. Wu, J. Zeng, P. Luo, J. Zhang, A. Geiger, Y. Qiao, and H. Li, "Generalized predictive model for autonomous driving," in *CVPR*, 2024.
- [41] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. M. Wolff, A. H. Lang, L. Fletcher, O. Beijbom, and S. Omari, "nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles," *arXiv preprint arXiv:2106.11810*, 2021.
- [42] H. Tian, T. Li, H. Liu, J. Yang, Y. Qiu, G. Li, J. Wang, Y. Gao, Z. Zhang, L. Wang, H. Ye, T. Tan, L. Chen, and H. Li, "SimScale: Learning to drive via real-world simulation at scale," in *CVPR*, 2026.
- [43] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.
- [44] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *IEEE Intelligent Vehicles Symposium (IV)*, 2015.
- [45] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.